

Language Models are Unsupervised Multitask Learners

这道题就是实现一个简易的Transformer解码器模块。对我来说最难的还是跟踪每个输入输出矩阵的形状。搞清楚形状后就会简单很多。这里的思路也是很简单，把每个步骤拆成一个算子，然后在main中拼接起来。其实按照题目给的信息和顺序，一步一步写就可以了。有一个比较难的就是Attention的拆分和组装，这里确实卡了很久。但是画个图就能大概知道逻辑。

接下来我根据代码和题目讲一下思路。

第一部分，我们需要计算Layer Norm1.

所以很显然，我们要写一个LN算子。LN算子接收一个x输入矩阵，这里的x的形状是(seq_len, d_model)，然后我们需要进行一些数学计算获取被乘在gamma前面的系数。也就是我们还需要计算mean和方差，最后根据数学公式组装起来即可。

这里的mean和方差是针对一行来计算的，一行的长度就是d_model，总共有seq_len行，所以我们需要用blockIdx.x来获取当前的行，然后用一个block计算一行，一个block中的一个thread计算这个元素。

我们想让一个block中的所有线程一起合作计算出mean，也就是先计算和。我们需要一个共享的变量part_sum，part就是说是这一行。这里我们使用了一个数组。因为这样可以类似归并来求和，提高效率。总共想要弄256个线程，所以数组就是256的长度。还需要mean和k，他们都是我们要计算的东西。后面就是让各个线程合作起来求和就行，然后除以d_model就是mean了。计算平方差的和也是完全同理的，知识步骤多一点。我们拿到了mean和k，前面的整个系数就是完全知道的了，把整个前面的结果算出来，然后和gamma[i]相乘，加上beta[i]，就是我们要的输出了，写入到out中。这里的out的形状是(seq_len, d_model)

代码是

```
__global__ void LN(const float *x, const float *gamma, const float *beta, float
*out) {
    // 我们让一个block处理一行
    // 一行就是d_model个数字
    // 然后有seq_len行

    // row代表当前是第几个block，也就是第几行
    int row = blockIdx.x;
    // thread_idx代表当前是这个block的第几个thread，也就是第几个元素
    int thread_idx = threadIdx.x;
    // current_start就是用row算出当前block的第一个元素从哪里开始
    int current_start = row * 768;

    /* 这个part_sum是后面用来存一行的数字的和的
    * 因为可以让每个thread只算3个
    * 这样256个thread就可以把这d_model个数字算好
    * 最后合并乘一个即可
    */
    __shared__ float part_sum[256];
    // 这个mean代表的是这行的平均数，就是那个u
    __shared__ float mean;
    // k代表的是1/sqrt(sigma + epsilon)，也就是前面的系数
    __shared__ float k;
```

```
float sum = 0.0f;
// 计算部分和
for(int i = thread_idx; i < d_model; i += 256) {
    sum += x[current_start + i];
}

part_sum[thread_idx] = sum;

// 然后这一步需要大家一起算完才可以继续
__syncthreads();

// 这边是合作求和
for(int i = 128; i > 0; i /= 2) {
    if(thread_idx < i) {
        part_sum[thread_idx] += part_sum[thread_idx + i];
    }
    __syncthreads();
}

// 求和完毕，全部合并到了partsum[0]那里了
if(thread_idx == 0) {
    mean = part_sum[0] / (float)d_model;
}

__syncthreads();

// 现在才开始计算sigma
// 先计算平方差的和

float squared_sum = 0.0f;
for(int i = thread_idx; i < d_model; i += 256) {
    float a = x[current_start + i] - mean;
    squared_sum += a * a;
}

// 这一步，直接复用之前的part_sum
// 因为之前的已经没有用了
part_sum[thread_idx] = squared_sum;
__syncthreads();

// 这一步和上面的一样。都是合作求和
for(int i = 128; i > 0; i /= 2) {
    if(thread_idx < i) {
        part_sum[thread_idx] += part_sum[thread_idx + i];
    }
    __syncthreads();
}

if(thread_idx == 0) {
    float mean_sum = part_sum[0] / (float)d_model;
    k = rsqrt(mean_sum + 1e-5f);
}
```

```

__syncthreads();

// mean和k都是shared的，大家都看得见
// 这一步就是计算最后的out了
for(int i = thread_idx; i < d_model; i+= 256) {
    out[current_start + i] = (x[current_start + i] - mean) * k * gamma[i] +
    beta[i];
}
}

```

第二部分，我们要计算QKV投影

这一步反而是非常简单的，只需要写一个矩阵相乘再加偏置的算子即可，所以我写了QKV算子，直接用前面那道矩阵乘法的修改一下就可以用了，现在的qkv矩阵形状是(seq_len, 3 * d_model),**这里要非常小心计算出来的qkv其实是三个矩阵竖着拼起来的，我们后面还得给他们拆开成Q,K,V**

代码是

```

// 就是普通的矩阵乘法
__global__ void QKV(const float *A, const float *B, const float *bias, float *C,
int M, int N, int K) {

    // 计算 C = A * B + bias

    // 根据block和thread找到现在要计算的row和col
    int row = blockDim.y * blockIdx.y + threadIdx.y;
    int col = blockDim.x * blockIdx.x + threadIdx.x;

    // 然后还需要判断一下row, col在不在范围内
    if(row >= M || col >= N) {
        return;
    }

    // 也就是我们要算的是C[row][col]
    float total = 0.0f;

    for(int x = 0; x < K; x++) {
        total += (A[row * K + x]) * (B[x * N + col]);
    }

    // 这里得转化一下half和float
    C[row * N + col] = total + bias[col];
}

```

第三部分，最难的，多头注意力

这一步非常费劲，以至于我一个算子都很难把逻辑写完整，所以我拆成了四个，分别是split_qkv,attention_1,attention_2, attention_3。

split_qkv就是前面我说的，来拆开qkv矩阵的那个算子，其实也就是把qvk的(row, col)给搬运到Q(row, col),qkv的

(row, col + d_model) 给搬运到K(row, col),qkv的(row, col + 2 * d_model) 给搬运到V(row, col)。其实不算难，代码实现就是

```
int row = idx / d_model;
int col = idx % d_model;
int source = row * 3 * d_model + col;

Q[idx] = qkv[source];
K[idx] = qkv[source + d_model];
V[idx] = qkv[source + 2 * d_model];
```

现在的Q,K,V形状都是(seq_len, d_model)

接下来要把Q,K,V拆成12个(seq_len, 64),但是这里继续拆分不太好，还不如直接用坐标偏移量来计算。

我们现在引入blockIdx.z来当作头的i，i的取值就是0到11，对应12个头。

那么第head个的Q[row][j]他就是Q[row * d_model + head * 64 + j],第head个的K[col][j]就是K[col * d_model + head * 64 + j]。

然后这里刚好是Qi乘以Ki的转置，那么我们也就不用竖着访问Ki了，直接横着来。也就是说，我们要的就是Q[row][j] * K[col][j]，j遍历64即可。

计算出out，out的形状现在是(seq_len, seq_len, 12) 如果想知道第head个头的out[row][col]，就是out[head * seq_len * seq_len + row * seq_len + col]

这里还要除以根号下dk，也就是除以8

然后我们来计算softmax，softmax就是把这一行的所有数的指数算出来，然后加起来作为分母，自己的指数作为分子。所以我们这里沿用前面计算mean的方法，合作求和。只是要多一步，就是用expf计算指数。现在的输出形状依旧是(seq_len, seq_len, 12)

最后就是乘以Vi矩阵了，这一步我们依旧用head代表当前是第几个头，然后头只会影响col，不影响row。

可以知道S[row][j]就是S[head * seq_len * seq_len + row * seq_len + j],V[j][col]就是V[j * d_model + head * 64 + col]，所以就这两个相乘，遍历j从0到seq_len即可算出A[row][col]

这里贴出四个算子

```
__global__ void split_qkv(const float *qkv, float *Q, float *K, float *V, int
seq_len) {
    int idx = blockDim.x * blockIdx.x + threadIdx.x;
    if(idx >= seq_len * d_model) {
        return;
    }

    // 也就是把qkv的row行col列搬运到Q的row行col列
    // 把qkv的row行col+d_model列搬运到Q的row行col列
    // 把qkv的row行col + 2 * d_model列搬运到Q的row行col列

    int row = idx / d_model;
    int col = idx % d_model;
    int source = row * 3 * d_model + col;

    Q[idx] = qkv[source];
```

```

    K[idx] = qkv[source + d_model];
    V[idx] = qkv[source + 2 * d_model];
}

__global__ void attention_1(float *Q, float *K, int seq_len, float *out) {

    // 这里的row和col都是输出的矩阵的row和col
    // 不是Q, K的
    int row = blockDim.y * blockIdx.y + threadIdx.y;
    int col = blockDim.x * blockIdx.x + threadIdx.x;

    if(row >= seq_len || col >= seq_len) {
        return;
    }

    // 这里的head代表当前是第几个头，一共12个
    // start 代表当前的头是从第几个元素开始的
    int head = blockIdx.z;
    int start = head * 64;

    float sum = 0.0f;

    // out[row][col] = Q[row][j] * K[col][j] (这里转置了)
    // Q[row][j] = Q[row * d_model + head * 64 + j]
    // K[col][j] = K[col * d_model + head * 64 + j]
    for(int j = 0; j < 64; j++) {
        sum += Q[row * d_model + head * 64 + j] * K[col * 768 + head * 64 + j];
    }

    // out的形状是12 * seq_len * seq_len
    out[head * seq_len * seq_len + row * seq_len + col] = sum / 8.0f;
}

__global__ void attention_2(float *S, int seq_len) {
    int head = blockIdx.z;
    int row = blockIdx.x;
    int thread_idx = threadIdx.x;

    /* 这里是因为
     * 现在的形状是(12, seq_len, seq_len)
     * 所以用head乘以两个seq_len，加上当前行
     * 最后的j遍历各个列
     */
    int current_start = head * seq_len * seq_len + row * seq_len;

    __shared__ float part_sum[256];

    float sum = 0.0f;
    for(int col = thread_idx; col < seq_len; col += 256) {
        // 用expf算e的指数
        float value = expf(S[current_start + col]);
        // 然后给S，相当于把S每个元素换成指数

```

```

        S[current_start + col] = value;
        // 顺便计算指数和
        sum += value;
    }

    part_sum[thread_idx] = sum;
    __syncthreads();

    // 后面依旧是合作求和
    for(int i = 128; i > 0; i /= 2) {
        if(thread_idx < i) {
            part_sum[thread_idx] += part_sum[thread_idx + i];
        }
        __syncthreads();
    }

    // 这里的意思是分母，也就是指数和
    float mother = part_sum[0];

    for(int col = thread_idx; col < seq_len; col += 256) {
        S[current_start + col] /= mother;
    }
}

__global__ void attention3(float *S, const float *V, int seq_len, float *A) {
    int row = blockDim.y * blockIdx.y + threadIdx.y;
    int col = blockDim.x * blockIdx.x + threadIdx.x;
    int head = blockIdx.z;

    if(row >= seq_len || col >= 64) return;

    int current_start = head * seq_len * seq_len + row * seq_len;
    int Vcol = head * 64 + col;

    float sum = 0.0f;
    // A[row][col] = S[row][j] * V[j][col]
    for(int j = 0; j < seq_len; j++) {
        sum += S[current_start + j] * V[j * d_model + Vcol];
    }

    A[row * d_model + Vcol] = sum;
}

```

第四部分，输出投影

这一部分又简单很多了，因为他依旧是矩阵的乘法和加法，我们都可以直接复用QKV那个算子，不需要额外写。所以我就直接用QKV算子了，传入的M,N,K分别是seq_len,d_model,d_model。

第五部分，残差连接1

这一部分更加简单，就是矩阵加法，算子非常简洁

```
__global__ void residual_add(const float* x, const float* P, float* out, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        out[idx] = x[idx] + P[idx];
    }
}
```

第六部分，LN2

跟LN1几乎一模一样，只是传进去的参数不太一样，用weights偏移量就可以

第七部分，前馈网络

其实拆开来看就是一个GELU和两个矩阵乘法和加法。矩阵乘法和加法完全可以复用QKV的算子，而GELU则需要另外写一个算子。其实GELU也非常简单，都只是一些普通的数学运算而已，这里我们直接把根号下派分之2换成数字0.7978845608f即可。所以代码就是

```
__global__ void gelu(float *data, int n) {
    int idx = blockDim.x * blockIdx.x + threadIdx.x;
    if(idx >= n) return;

    float x = data[idx];
    data[idx] = 0.5f * x * (1.0f + tanhf(0.7978845608f * (x + 0.044715f * x * x * x * x)));
}
```

第八部分，残差连接2

完全复用残差连接1的算子即可

代码部分和结果

这样子整个的代码就写完了。

在这里贴出代码，代码中有大量的注释

```
#include <cuda_runtime.h>
#define d_model 768

__global__ void LN(const float *x, const float *gamma, const float *beta, float *out) {
    // 我们让一个block处理一行
    // 一行就是d_model个数字
    // 然后有seq_len行

    // row代表当前是第几个block，也就是第几行
    int row = blockIdx.x;
    // thread_idx代表当前是这个block的第几个thread，也就是第几个元素
```

```
int thread_idx = threadIdx.x;
// current_start就是用row算出当前block的第一个元素从哪里开始
int current_start = row * 768;

/* 这个part_sum是后面用来存一行的数字的和的
 * 因为可以让每个thread只算3个
 * 这样256个thread就可以把这d_model个数字算好
 * 最后合并乘一个即可
 */
__shared__ float part_sum[256];
// 这个mean代表的是这行的平均数，就是那个u
__shared__ float mean;
// k代表的是1/sqrt(sigma + epsilon)，也就是前面的系数
__shared__ float k;

float sum = 0.0f;
// 计算部分和
for(int i = thread_idx; i < d_model; i += 256) {
    sum += x[current_start + i];
}

part_sum[thread_idx] = sum;

// 然后这一步需要大家一起算完才可以继续
__syncthreads();

// 这边是合作求和
for(int i = 128; i > 0; i /= 2) {
    if(thread_idx < i) {
        part_sum[thread_idx] += part_sum[thread_idx + i];
    }
    __syncthreads();
}

// 求和完毕，全部合并到了partsum[0]那里了
if(thread_idx == 0) {
    mean = part_sum[0] / (float)d_model;
}

__syncthreads();

// 现在才开始计算sigma
// 先计算平方差的和

float squared_sum = 0.0f;
for(int i = thread_idx; i < d_model; i += 256) {
    float a = x[current_start + i] - mean;
    squared_sum += a * a;
}

// 这一步，直接复用之前的part_sum
// 因为之前的已经没有用了
part_sum[thread_idx] = squared_sum;
```



```

__syncthreads();

// 这一步和上面的一样。都是合作求和
for(int i = 128; i > 0; i /= 2) {
    if(thread_idx < i) {
        part_sum[thread_idx] += part_sum[thread_idx + i];
    }
    __syncthreads();
}

if(thread_idx == 0) {
    float mean_sum = part_sum[0] / (float)d_model;
    k = rsqrt(mean_sum + 1e-5f);
}

__syncthreads();

// mean和k都是shared的，大家都看得见
// 这一步就是计算最后的out了
for(int i = thread_idx; i < d_model; i+= 256) {
    out[current_start + i] = (x[current_start + i] - mean) * k * gamma[i] +
beta[i];
}
}

// 就是普通的矩阵乘法
__global__ void QKV(const float *A, const float *B, const float *bias, float *C,
int M, int N, int K) {

    // 计算 C = A * B + bias

    // 根据block和thread找到现在要计算的row和col
    int row = blockDim.y * blockIdx.y + threadIdx.y;
    int col = blockDim.x * blockIdx.x + threadIdx.x;

    // 然后还需要判断一下row, col在不在范围内
    if(row >= M || col >= N) {
        return;
    }

    // 也就是我们要算的是C[row][col]
    float total = 0.0f;

    for(int x = 0; x < K; x++) {
        total += (A[row * K + x]) * (B[x * N + col]);
    }

    // 这里得转化一下half和float
    C[row * N + col] = total + bias[col];
}

__global__ void split_qkv(const float *qkv, float *Q, float *K, float *V, int

```

```

seq_len) {
    int idx = blockDim.x * blockIdx.x + threadIdx.x;
    if(idx >= seq_len * d_model) {
        return;
    }

    // 也就是把qkv的row行col列搬运到
    // Q的row行col列
    // K的row行col + d_model列
    // V的row行col + 2 * d_model列

    int row = idx / d_model;
    int col = idx % d_model;
    int source = row * 3 * d_model + col;

    Q[idx] = qkv[source];
    K[idx] = qkv[source + d_model];
    V[idx] = qkv[source + 2 * d_model];
}

__global__ void attention_1(float *Q, float *K, int seq_len, float *out) {

    // 这里的row和col都是输出的矩阵的row和col
    // 不是Q, K的
    int row = blockDim.y * blockIdx.y + threadIdx.y;
    int col = blockDim.x * blockIdx.x + threadIdx.x;

    if(row >= seq_len || col >= seq_len) {
        return;
    }

    // 这里的head代表当前是第几个头，一共12个
    int head = blockIdx.z;

    float sum = 0.0f;

    // out[row][col] = Q[row][j] * K[col][j] (这里转置了)
    // Q[row][j] = Q[row * d_model + head * 64 + j]
    // K[col][j] = K[col * d_model + head * 64 + j]
    for(int j = 0; j < 64; j++) {
        sum += Q[row * d_model + head * 64 + j] * K[col * 768 + head * 64 + j];
    }

    // out的形状是12 * seq_len * seq_len
    out[head * seq_len * seq_len + row * seq_len + col] = sum / 8.0f;
}

__global__ void attention_2(float *S, int seq_len) {
    int head = blockIdx.z;
    int row = blockIdx.x;
    int thread_idx = threadIdx.x;

```

```

/* 这里是因为
 * 现在的形状是(seq_len, seq_len)
 * 所以用head乘以两个seq_len, 加上当前行
 * 最后的j遍历各个列
 */
int current_start = head * seq_len * seq_len + row * seq_len;

__shared__ float part_sum[256];

float sum = 0.0f;
for(int col = thread_idx; col < seq_len; col += 256) {
    // 用expf算e的指数
    float value = expf(S[current_start + col]);
    // 然后给S, 相当于把S每个元素换成指数
    S[current_start + col] = value;
    // 顺便计算指数和
    sum += value;
}

part_sum[thread_idx] = sum;
__syncthreads();

// 后面依旧是合作求和
for(int i = 128; i > 0; i /= 2) {
    if(thread_idx < i) {
        part_sum[thread_idx] += part_sum[thread_idx + i];
    }
    __syncthreads();
}

// 这里的意思是分母, 也就是指数和
float mother = part_sum[0];

for(int col = thread_idx; col < seq_len; col += 256) {
    S[current_start + col] /= mother;
}
}

__global__ void attention3(float *S, const float *V, int seq_len, float *A) {
    int row = blockDim.y * blockIdx.y + threadIdx.y;
    int col = blockDim.x * blockIdx.x + threadIdx.x;
    int head = blockIdx.z;

    if(row >= seq_len || col >= 64) return;

    int current_start = head * seq_len * seq_len + row * seq_len;
    int Vcol = head * 64 + col;

    float sum = 0.0f;
    // A[row][col] = S[row][j] * V[j][col]
    for(int j = 0; j < seq_len; j++) {
        sum += S[current_start + j] * V[j * d_model + Vcol];
    }
}

```

```

    A[row * d_model + Vcol] = sum;
}

__global__ void residual_add(const float* x, const float* P, float* out, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        out[idx] = x[idx] + P[idx];
    }
}

__global__ void gelu(float *data, int n) {
    int idx = blockDim.x * blockIdx.x + threadIdx.x;
    if(idx >= n) return;

    float x = data[idx];
    data[idx] = 0.5f * x * (1.0f + tanhf(0.7978845608f * (x + 0.044715f * x * x *
x)))));
}

extern "C" void solve(const float* x, float* output, const float* weights, int
seq_len) {
    // 把这里改成你的实现
    float *x_norm;
    // 这里声明在显存上
    // 因为他的输出还是GPU继续使用的
    // 不要搬到CPU再搬过去GPU, 浪费
    cudaMalloc(&x_norm, (size_t)seq_len * d_model * sizeof(float));

    // 第一步, 计算LN层的结果
    // 输入的x是(seq_len, d_model)
    const float *gamma1 = weights;
    const float *beta1 = weights + 768;
    LN<<<seq_len, 256>>>(x, gamma1, beta1, x_norm);
    // 经过layernorm以后是(seq_len, d_model)

    // 第二部计算QKV矩阵
    float *qkv;
    // 因为需要生成三个向量QKV, 每份都是d_model个
    cudaMalloc(&qkv, (size_t)seq_len * 3 * d_model * sizeof(float));

    dim3 block(16, 16);
    dim3 grid((3 * d_model + block.x - 1) / block.x, (seq_len + block.y - 1) /
block.y);

    const float *Wqkv = weights + 1536;
    const float *Bqkv = weights + 1771008;
    QKV<<<grid, block>>>(x_norm, Wqkv, Bqkv, qkv, seq_len, 3 * d_model, d_model);

    // 释放掉不用的x_norm
    cudaFree(x_norm);
}

```

```

// 现在我们得拆分qkv为三个单独的矩阵了
float *Q, *K, *V;
cudaMalloc(&Q, (size_t)seq_len * d_model * sizeof(float));
cudaMalloc(&K, (size_t)seq_len * d_model * sizeof(float));
cudaMalloc(&V, (size_t)seq_len * d_model * sizeof(float));

int block2 = 256;
int grid2 = (seq_len * d_model + block2 - 1) / block2;
split_qkv<<<grid2, block2>>>(qkv, Q, K, V, seq_len);

// 释放掉不用的qkv矩阵
cudaFree(qkv);

// 现在开始完成多头注意力的部分，拆成三部分
dim3 block3(16, 16);
dim3 grid3((seq_len + block3.x - 1) / block3.x, ((seq_len) + block3.y - 1) /
block3.y, 12);

float *out1;
cudaMalloc(&out1, (size_t)12 * seq_len * seq_len * sizeof(float));
attention_1<<<grid3, block3>>>(Q, K, seq_len, out1);

// 释放掉不用的Q,K矩阵
cudaFree(Q);
cudaFree(K);

dim3 grid4(seq_len, 1, 12);
attention_2<<<grid4, 256>>>(out1, seq_len);

// 计算最后的A
float *A;
cudaMalloc(&A, (size_t)seq_len * d_model * sizeof(float));

dim3 block5(16, 16);
dim3 grid5((64 + block5.x - 1) / block5.x, (seq_len + block5.y - 1) /
block5.y, 12);

attention3<<<grid5, block5>>>(out1, V, seq_len, A);

// 释放掉不用的V矩阵
cudaFree(V);

// 释放掉不用的out1矩阵
cudaFree(out1);

// 到这里多头注意力计算好了
// 后面我们计算输出投影
// 我直接复用QKV，因为他们是完全一样的
const float *Wattn = weights + 1773312;
const float *Battn = weights + 2363136;
dim3 block6(16, 16);
dim3 grid6((d_model + block6.x - 1) / block6.x, (seq_len + block6.y - 1) /
block6.y);

```

```

float *P;
cudaMalloc(&P, (size_t)seq_len * d_model * sizeof(float));
QKV<<<grid6, block6>>>(A, W attn, B attn, P, seq_len, d_model, d_model);

// 释放掉不用的A矩阵
cudaFree(A);

// 残差连接, 最简单的
float *new_x;
cudaMalloc(&new_x, (size_t)seq_len * d_model * sizeof(float));
int n = seq_len * d_model;
residual_add<<<(n + 255) / 256, 256>>>(x, P, new_x, n);

// 释放掉不用的P矩阵
cudaFree(P);

// 然后来一层LN2(x)
float *h_norm;
cudaMalloc(&h_norm, (size_t)seq_len * d_model * sizeof(float));
const float *gamma2 = weights + 2363904;
const float *beta2 = weights + 2364672;
LN<<<seq_len, 256>>>(new_x, gamma2, beta2, h_norm);

/* 前馈网络可以拆分
 * 可以拆分成QKV的h_norm
 * 用GELU计算
 * 再把结果经过一个QKV
 */

// 注意到ffn_dim = 3,072
dim3 block_ff(16, 16);
dim3 grid_fc((3072 + block_ff.x - 1) / block_ff.x, (seq_len + block_ff.y - 1)
/ block_ff.y);

const float *Wfc = weights + 2365440;
const float *Bfc = weights + 4724736;
float *out2;
cudaMalloc(&out2, (size_t)seq_len * 3072 * sizeof(float));
QKV<<<grid_fc, block_ff>>>(h_norm, Wfc, Bfc, out2, seq_len, 3072, d_model);

// 这里是对out2的每个元素应用GELU
int n_gelu = seq_len * 3072;
gelu<<<(n_gelu + 255) / 256, 256>>>(out2, n_gelu);

// 最后是再一个QKV
dim3 grid_proj((d_model + block_ff.x - 1) / block_ff.x, (seq_len + block_ff.y
- 1) / block_ff.y);

const float *Wproj = weights + 4727808;
const float *Bproj = weights + 7087104;
float *F;
cudaMalloc(&F, (size_t)seq_len * d_model * sizeof(float));

```

```

    QKV<<<grid_proj, block_ff>>>(out2, Wproj, Bproj, F, seq_len, d_model, 3072);

    // 最后再来一个残差连接
    float *output_on_gpu;
    cudaMalloc(&output_on_gpu, (size_t)seq_len * d_model * sizeof(float));
    residual_add<<<(n + 255) / 256, 256>>>(new_x, F, output_on_gpu, n);

    cudaMemcpy(output, output_on_gpu, (size_t)seq_len * d_model * sizeof(float),
cudaMemcpyDeviceToHost);

    cudaFree(new_x);
    cudaFree(h_norm);
    cudaFree(out2);
    cudaFree(F);
    cudaFree(output_on_gpu);
}

```

测试一下

```

nsccl@login:~$ nsccl-run bash run_tests.sh
execution 12fffd288-b1b1-4769-937e-0128731dcbf2
ok   example_seq4: seq=4 time_ms=0.662
ok   single_token: seq=1 time_ms=0.355
ok   zero_x_seq4: seq=4 time_ms=0.352
ok   seq2: seq=2 time_ms=0.350
ok   seq4: seq=4 time_ms=0.351
ok   seq16: seq=16 time_ms=0.423
ok   seq64: seq=64 time_ms=0.754
ok   seq30: seq=30 time_ms=0.479
ok   seq100: seq=100 time_ms=1.452
ok   seq128: seq=128 time_ms=1.834
ok   seq256: seq=256 time_ms=3.826
ok   perf_seq1024: seq=1024 time_ms=20.652
summary cases=12 failures=0 worst_case_ms=20.652
cuda-gpt2-block_RESULT: PASS
WARNING: Overriding HOME environment variable with APPTAINERENV_HOME is not
permitted
execution 12fffd288-b1b1-4769-937e-0128731dcbf2: completed

```

性能优化上,我估计我这个是有很大大优化空间的。首先就是,我声明了太多的cudaMalloc,然后没有复用这部分,所以浪费了不少显存。而且多了很多搬运的过程。然后就是,现在我选择的block都是(16, 16),这可能也不是利用率最高的方式,还有就是这些矩阵乘法基本没有做优化。

还有一个超级奇怪的地方。我昨天没有写cudaFree,测试的结果很明显比今天写了cudaFree的快了很多,太神奇了,下面贴出的是没有写cudaFree的版本的测试结果

```

execution a6602a30-5664-449c-a5e6-dc84cd6befb3
ok   example_seq4: seq=4 time_ms=0.571

```

```
ok   single_token: seq=1 time_ms=0.278
ok   zero_x_seq4: seq=4 time_ms=0.281
ok   seq2: seq=2 time_ms=0.279
ok   seq4: seq=4 time_ms=0.282
ok   seq16: seq=16 time_ms=0.350
ok   seq64: seq=64 time_ms=0.679
ok   seq30: seq=30 time_ms=0.492
ok   seq100: seq=100 time_ms=1.146
ok   seq128: seq=128 time_ms=1.406
ok   seq256: seq=256 time_ms=2.681
ok   perf_seq1024: seq=1024 time_ms=13.467
summary cases=12 failures=0 worst_case_ms=13.467
cuda-gpt2-block_RESULT: PASS
WARNING: Overriding HOME environment variable with APPTAINERENV_HOME is not
permitted
solution.cu(152): warning #177-D: variable "start" was declared but never
referenced
    int start = head * 64;
        ^

Remark: The warnings can be suppressed with "-diag-suppress <warning-number>"

execution a6602a30-5664-449c-a5e6-dc84cd6befb3: completed
```

明显快了不少